

STARTUP FILE & LINKER SCRIPT

From Power-On to main()

A line-by-line walkthrough of startup_stm32f103.s and STM32F103C8.ld

Two Files, One Job

Before a single line of your C code can run, something has to tell the CPU where the stack begins and what to execute first — and make sure global variables hold sane values. On bare metal, you provide both.

S

Startup File

`startup_stm32f103.s`

- Defines the vector table — the hardware-mandated array of addresses at the base of FLASH
- Implements `Reset_Handler`: the very first code to run after reset
- Copies `.data` from FLASH to RAM and zero-fills `.bss` before `main()` is safe to call

L

Linker Script

`STM32F103C8.ld`

- Declares where FLASH and RAM physically live on this exact chip
- Places the vector table, code, and constants into FLASH
- Splits variables into `.data` (needs a FLASH-stored initial value) and `.bss` (just needs zero), and exports the boundary symbols the startup file needs

They are tightly coupled: the startup file references symbols that only exist because the linker script defines them.

STM32F103C8T6 Memory Map

FLASH — Code Storage

0x08000000 – 0x0800FFFF (64 KB)

.isr_vector → .text → .rodata → initial values for .data

RAM — Working Memory

0x20005000 (top) = _estack

Initial stack pointer — stack grows downward ↓

... **stack space** ...

.bss

Zero-initialized globals

.data

Initialized globals (writable RAM copies)

0x20000000 (bottom, 20 KB total)

0x08000000 and 0x20000000 are fixed by ARM's Cortex-M memory map convention and ST's sizing for the “C8” density variant (64 KB FLASH / 20 KB RAM).

ENTRY and the MEMORY Block

```
ENTRY(Reset_Handler)
```

```
MEMORY
```

```
{
```

```
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 64K
```

```
    RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 20K
```

```
}
```

- ENTRY doesn't control boot electrically — Cortex-M3 always boots via the vector table. It sets the ELF header's entry field for debuggers and tools like objdump/gdb.
- ORIGIN / LENGTH come straight from the chip's datasheet for each named region.
- (rx) on FLASH — readable + executable, not writable. Flash can't be written like a variable; it needs a special programming sequence.
- (rwx) on RAM — readable, writable, executable. Variables that change at runtime must live in writable memory.

`_estack` and `.isr_vector` Placement

```
_estack = ORIGIN(RAM) + LENGTH(RAM);
```

- $0x20000000 + 0x5000 = 0x20005000$ — one past the last valid RAM byte.
- ARM stacks grow downward, so “empty” SP starts at the top of RAM.
- Consumed by the startup file as word 0 of the vector table — the value the CPU loads into SP at power-on.

```
.isr_vector :  
{  
    KEEP(*(.isr_vector))  
} > FLASH
```

- Placed at $0x08000000$ — not convention, a hardware requirement. Cortex-M3 reads word 0 (SP) and word 1 (reset vector) from this exact address on reset.
- `KEEP()` matters because nothing in C “calls” this table by name — without it, `--gc-sections` would discard it as unused.

.text and .data — the FLASH/RAM Trick

```
.text :
{
    *(.text*)
    *(.rodata*)
} > FLASH

_sidata = LOADADDR(.data);
.data :
{
    _sdata = .;  *(.data*)  _edata = .;
} > RAM AT> FLASH
```

Initialized globals need to be writable (RAM) but must start with a real value (stored in FLASH). “> RAM AT> FLASH” gives the section a runtime address in RAM and a load address in FLASH — both at once.

_sdata

Initial values' location in FLASH (copy source)

_sdata

Start of .data in RAM (copy destination start)

_edata

End of .data in RAM (copy destination end)

These symbols do nothing by themselves — Reset_Handler must explicitly copy FLASH → RAM using them.

.bss — Zero-Initialized Globals

```
.bss :  
{  
    _sbss = .;  
    * (.bss*)  
    * (COMMON)  
    _ebss = .;  
} > RAM
```

- Handles globals with no initializer (int total;) — C guarantees these start at zero.
- No reason to waste FLASH storing zeros — the linker just reserves RAM space and records `_sbss` / `_ebss`.
- `*(COMMON)` catches old-style “tentative definitions” some toolchains group separately from ordinary `.bss`.
- Nothing hardware-level zeroes this region — like `.data`, `Reset_Handler` must do it explicitly before `main()` runs.

The Vector Table

```
g_pfnVectors:  
    .word _estack          /* SP      */  
    .word Reset_Handler   /* Reset  */  
    .word Default_Handler /* NMI    */  
    .word Default_Handler /* HardFault */  
    ...
```

The remaining entries are core exceptions and, in a real project, the full peripheral IRQ list — here they all point at Default_Handler as placeholders.

1

Word 0 — Initial SP

The Cortex-M3 reset sequence, done entirely in hardware, loads this value directly into the SP register — before any code has run.

2

Word 1 — Reset Vector

The address the core jumps to next: Reset_Handler. Bit 0 is set to mark it as Thumb code, handled automatically by the assembler/linker.

Reset_Handler — Copy .data to RAM

```
ldr r0, =_sidata
ldr r1, =_sdata
ldr r2, =_edata
copy_data_loop:
    cmp r1, r2
    bge copy_data_done
    ldr r3, [r0], #4
    str r3, [r1], #4
    b copy_data_loop
copy_data_done:
```

- r0 walks FLASH from _sidata (stored initial values); r1 walks RAM from _sdata (where live variables belong).
- One 32-bit word copied per loop iteration: ldr post-increments r0, str post-increments r1.
- Loop continues until r1 reaches _edata.
- After this, every initialized global variable holds the value your C source specified.

Zero .bss, Then Call main()

```
ldr r1, =_sbss
ldr r2, =_ebss
movs r3, #0
zero_bss_loop:
    cmp r1, r2
    bge zero_bss_done
    str r3, [r1], #4
    b zero_bss_loop
zero_bss_done:

bl main
b .
```

- Same copy pattern, simpler: no source data, just write constant 0 across `_sbss`–`_ebss`.
- Guarantees every uninitialized global starts at zero, as C requires — RAM otherwise holds unpredictable leftover contents.

Only now is it safe to enter C

Stack is set by hardware, `.data` is copied, `.bss` is zeroed — every global holds a value C code is entitled to assume it has. `bl main` hands off control; the trailing `b .` traps the core if `main()` ever returns, which it isn't meant to on a freestanding target.

How the Two Files Connect

Symbol	Defined In	Used In	Purpose
<code>_estack</code>	Linker script	Startup file (vector table word 0)	Initial stack pointer
<code>Reset_Handler</code>	Startup file	Linker (ENTRY) & startup (vector table word 1)	First code after reset
<code>_sidata / _sdata / _edata</code>	Linker script	Startup file (.data copy loop)	FLASH→RAM copy bounds
<code>_sbss / _ebss</code>	Linker script	Startup file (.bss zero loop)	Zero-fill bounds
<code>.isr_vector</code> placement	Linker script	Hardware (reads 0x08000000 directly)	Boot-time SP & reset address

Neither file compiles into anything meaningful alone: the startup file's symbol references are undefined without the linker script, and the linker script's layout is pointless without `Reset_Handler` to act on it.

Everything C Assumes, Someone Has to Set Up First

- Hardware sets the stack pointer from word 0 of the vector table — before any instruction runs.
- Reset_Handler copies .data from FLASH to RAM and zero-fills .bss — before main() is entered.
- Only after both steps does every global variable hold the value C code is entitled to assume it has.

This example truncates the IRQ table to core exceptions only and skips clock configuration (SystemInit / HSE / PLL) — a production project extends both.